analogy	service method		
Main risk	Large blast radius	Data dump, leaking sensitive data	Vague or unevaluated instructions

Nuance: A tool is not always a write. `get_pod_logs` is read-only but still a tool, because the model *chooses* to fetch it with parameters. A resource is context the client attaches by URI, and no model decision is involved.

2. The Decision Tree

1. **Does the model need to *do* something in a real system** (query with parameters, create, restart, send)? Use a **Tool**.
2. **Does it need to *read* something it can't answer without** (a file, schema, manifest, document)? Use a **Resource**.
3. **Does a task repeat and need the same reasoning frame every time** (review a PR, summarize an incident, write release notes)? Use a **Prompt**.
4. **Real tasks usually need all three.**

Worked example: "Why is my app failing?"

1. The client attaches the deployment YAML as a **resource**.
2. The model calls `get_pod_logs` (a tool) to pull recent logs.
3. The user (or client) invokes the `incident-analysis` **prompt**, which structures the final answer (symptoms, evidence, likely cause, next step).

answer (symptoms, evidence, likely cause, next step).

3. Mapping to Spring Boot Concepts

MCP concept	Spring Boot / Spring AI concept	Notes
MCP server	Spring Boot app with the MCP server starter	Transport: STDIO (local) or Streamable HTTP/SSE (remote, e.g. ECS Fargate)
Tool	A `@Component` method annotated `@McpTool`	Delegate to a `@Service`. Keep the method a thin ada
Tool input schema	Method parameters with `@McpToolParam`, generated as JSON Schema	Parameter descriptions are what the model reads, sc write them well
Resource	A `@McpResource` method keyed by URI	Behaves like a read-only `GET` over a read model
Resource template	URI with `{variables}` bound to method parameters	e.g. `k8s://namespaces/{namespace}/deployments/{n`
Prompt	A `@McpPrompt` method with `@McpArg` arguments	Returns a `GetPromptResult` made of messages, like template with parameters
Authorization	Spring Security, `@PreAuthorize` on tool methods	Different scopes for read tools vs mutating tools
Validation	Bean Validation (`@Validated`, `@Min`, `@Pattern`)	Reject bad input before it reaches a service
Resilience	Resilience4j circuit breaker, retry, time limiter around clients	Matches the circuit-breaker and retry lessons
Idempotency	Unique-key table or `INSERT ... ON CONFLICT`	Required for mutating tools
Observability	Micrometer metrics and tracing per tool, resource and prompt	Tag by primitive name and outcome
Config	`application.yml` (`spring.ai.mcp.server.*`)	Name, version, transport type

> **Version note:** The annotation names and packages below follow Spring AI's MCP annotations support. Package names and some attribute names have changed between Spring AI releases, so confirm them against the version your project uses before copying.

4. Example Domain: An Ops Assistant MCP Server

A small server that helps diagnose and fix problems in a Kubernetes-hosted application.

Primitive	Name	Purpose
Tool	`get_pod_logs`	Read-only. Fetch recent log lines for one pod
Tool	`restart_deployment`	Mutating. Restart one deployment, with approval and idempotency
Resource	`k8s://namespaces/{namespace}/deployments/{name}`	Read the deployment manifest (YAML)
Resource	`db://schemas/public`	Read the database schema summary
Prompt	`incident-analysis`	Structure an incident investigation
Prompt	`release-notes`	Turn a change list into release notes

5. Project Setup

Maven dependency

```
<dependency>
  <groupId>org.springframework.ai</groupId>
  <artifactId>spring-ai-starter-mcp-server-webmvc</artifactId>
</dependency>
```

`application.yml`

```
spring:
  ai:
    mcp:
      server:
        name: ops-mcp-server
        version: 1.0.0
        type: SYNC
        protocol: STREAMABLE   # remote HTTP transport; use STDIO for local-only serv
```

Suggested package layout

```
com.example.opsmcp
 ├─ OpsMcpApplication.java
 ├─ tools/      OpsTools.java            (@McpTool, thin adapters)
 ├─ resources/  OpsResources.java        (@McpResource)
 └─ prompts/    OpsPrompts.java          (@McpPrompt)
```

```
├─ prompts/    OpsPrompts.java          (@McpPrompt)
├─ service/    PodLogService, DeploymentService, ApprovalService
├─ security/   SecurityConfig
└─ persistence/IdempotencyRepository
```

Principle: The MCP layer is an *adapter*. Business logic stays in `service/`, so it can be tested and reused without MCP.

6. Tools: Actions

6.1 Read-only tool: `get_pod_logs`

```
@Component
public class OpsTools {

    private final PodLogService podLogs;
    private final DeploymentService deployments;

    public OpsTools(PodLogService podLogs, DeploymentService deployments) {
        this.podLogs = podLogs;
        this.deployments = deployments;
    }

    @McpTool(
        name = "get_pod_logs",
        description = """
            Fetch the most recent log lines for ONE pod. Read-only.
            Use when diagnosing a crash, error or restart loop.
            Not for metrics or cluster-wide search.
            """)
    public PodLogs getPodLogs(
            @McpToolParam(description = "Kubernetes namespace, e.g. 'payments'", requ
            String namespace,
            @McpToolParam(description = "Exact pod name", required = true)
            String pod,
            @McpToolParam(description = "Number of lines, 1–500. Default 100", requir
            Integer lines) {

        int n = (lines == null) ? 100 : Math.min(Math.max(lines, 1), 500); // hard ca
        return podLogs.tail(namespace, pod, n);
    }
}
```

Why this is a good tool

- One clear action with a narrow boundary (not `get_everything`).
- The description says what it does, when to use it, and when *not* to.
- Input is bounded (line cap) and typed, which limits blast radius and context bloat.

6.2 Mutating tool: `restart_deployment`

```
    @McpTool(
        name = "restart_deployment",
        description = """
            Rolling-restart ONE deployment. MUTATING: requires a valid approvalId
```

```
            Rolling restart ONE deployment. NOTHING. requires a valid approvalId.
            Safe to retry: the same idempotencyKey returns the original result.
            """)
    @PreAuthorize("hasAuthority('SCOPE_ops.write')")
    public RestartResult restartDeployment(
            @McpToolParam(description = "Namespace", required = true) String namespac
            @McpToolParam(description = "Deployment name", required = true) String de
            @McpToolParam(description = "Caller-generated unique key (UUID) for safe
            String idempotencyKey,
            @McpToolParam(description = "Human approval id from the approval workflow
            String approvalId) {

        return deployments.restartOnce(namespace, deployment, idempotencyKey, approva
    }
```

6.3 Behind the tool: idempotency in the service

```
@Service
public class DeploymentService {

    private final IdempotencyRepository keys;
    private final ApprovalService approvals;
    private final KubernetesClient k8s;

    @Transactional
    public RestartResult restartOnce(String ns, String name, String key, String appro
        // 1. Return the stored result for a repeated key (safe retry)
        var existing = keys.findByKey(key);
        if (existing.isPresent()) return existing.get().result();

        // 2. Enforce the approval gate
        if (!approvals.isValid(approvalId, "restart", ns, name)) {
            return RestartResult.error("APPROVAL_INVALID",
                "Approval is missing, expired, or for a different target.",
                "ASK_USER");
        }

        // 3. Perform the action, then record the result under the key
        k8s.apps().deployments().inNamespace(ns).withName(name).rolling().restart();
        var result = RestartResult.ok(ns, name);
        keys.save(key, result);
        return result;
    }
}
```

Connection to the reliability lessons

- **Idempotency key** prevents a retry from restarting twice.
- The error returns a **stable code**, a clear message, and a **suggested action** (`ASK_USER`), so the model knows not to retry blindly.
- Wrap the Kubernetes client with a **circuit breaker** so an unavailable cluster doesn't fill the context with errors.

> Check how your Spring AI version maps thrown exceptions to MCP tool errors (`isError: true`). Returning a structured error payload, as above, keeps the behavior explicit.

7. Resources: Context

7.1 Deployment manifest as a URI template

```
@Component
public class OpsResources {

    private final DeploymentService deployments;
    private final SchemaService schemas;

    @McpResource(
        uri = "k8s://namespaces/{namespace}/deployments/{name}",
        name = "deployment-manifest",
        description = "Sanitized YAML manifest of one deployment. Secrets are redacte
        mimeType = "application/yaml")
    public ReadResourceResult deploymentManifest(
            ReadResourceRequest request, String namespace, String name) {

        String yaml = deployments.manifestYaml(namespace, name); // already redacted
        return new ReadResourceResult(List.of(
            new TextResourceContents(request.uri(), "application/yaml", yaml)));
    }

    @McpResource(
        uri = "db://schemas/public",
        name = "postgres-public-schema",
        description = "Table and column summary for the public schema. No row data.",
        mimeType = "text/markdown")
    public ReadResourceResult publicSchema(ReadResourceRequest request) {
        return new ReadResourceResult(List.of(
            new TextResourceContents(request.uri(), "text/markdown", schemas.publicSu
    }
}
```

7.2 Resource design rules (with Spring tools)

Rule	Spring implementation
Scoped and intentional	One URI per concept. Avoid `all-data` URIs
No secrets	Redact in the service layer, never in the MCP adapter
Bounded size	Truncate or summarize, and cap bytes
Cheap to re-read	`@Cacheable` with a short TTL where data is stable
Access-controlled	Spring Security, plus per-namespace checks in the service
Clear MIME type	`application/yaml`, `text/markdown`, `application/json`

Anti-pattern: A resource that returns the whole codebase or every customer record. That is noise and risk disguised as convenience.

8. Prompts: Workflows

8.1 Incident analysis prompt

```
@Component
public class OpsPrompts {

    @McpPrompt(
        name = "incident-analysis",
        description = "Structured investigation of a failing service.")
    public GetPromptResult incidentAnalysis(
            @McpArg(name = "service", description = "Service name", required = true)
            @McpArg(name = "namespace", description = "Kubernetes namespace", require

        String text = """
            You are investigating a failure in service '%s' (namespace '%s').

            Steps:
            1. Read the deployment manifest resource for this service.
            2. Call get_pod_logs for the affected pods (start with 100 lines).
            3. Do NOT call restart_deployment unless the user explicitly approves.

            Answer in this format:
            - Symptoms
            - Evidence (quote log lines or manifest fields)
            - Most likely cause
            - Recommended next step (and what you would NOT do yet)
            """.formatted(service, namespace);

        return new GetPromptResult("Incident analysis",
            List.of(new PromptMessage(Role.USER, new TextContent(text))));
    }
}
```

8.2 What makes a good prompt primitive

- It defines the **task**, the **arguments**, and the **output shape**.
- It references the relevant resources and tools by name, so the workflow is repeatable.
- It states guardrails (for example, "no restart without approval").
- Because it is versionable text, you can review it in a PR and evaluate it against test cases.

9. End-to-End Flow

Step	Primitive	What happens	Spring component
1	Prompt	User runs `incident-analysis` for `orders` in `payments`	`OpsPrompts`
2	Resource	Client attaches the deployment manifest	`OpsResources`
3	Tool	Model calls `get_pod_logs`	`OpsTools` then `PodLogService`

4	(Model)	Finds an OOMKilled loop and proposes a restart	n/a
5	Human	Approver grants an approval id	`ApprovalService`
6	Tool	Model calls `restart_deployment` with approval and idempotency key	`OpsTools` then `DeploymentService`

10. Common Mistakes and Spring Fixes

Mistake	Why it hurts	Fix
One `admin` or `run_anything` tool	Hidden intent, huge blast radius	Split into narrow, named tools with separate scopes
Tools that overlap (`search_docs` vs `find_documents`)	Inconsistent tool selection	Merge them or give each a distinct description
Using a tool just to read a static document	Wastes model turns and adds latency	Expose it as a resource
Resource that dumps everything	Noise, cost, data leakage	Scoped URIs, size caps, redaction
Prompt used as a hidden system policy	Not reviewable or testable	Version prompts in code and test them
Mutating tool without idempotency	Duplicate side effects on retry	Idempotency key table (section 6.3)
No timeouts or circuit breaker	Cascading failures, context filled with errors	Resilience4j around downstream clients
Returning errors as plain success text	Model proceeds as if it worked	Structured error with code, message, context, suggested action
Too many tools on one server per agent	Tool bloat, worse routing	Split into specialist servers or expose only role-relevant tools

11. Security and Governance Checklist

- ☐ Authenticate the MCP endpoint (OAuth2/JWT with Spring Security) when exposed over HTTP.
- ☐ Use separate scopes: `ops.read` for read tools and resources, `ops.write` for mutating tools.
- ☐ Apply `@PreAuthorize` and per-resource authorization in the service layer.
- ☐ Validate all tool input (`@Validated`, bounds, allow-lists of namespaces).
- ☐ Require human approval for destructive tools.
- ☐ Treat any text returned by downstream systems (logs, tickets, web pages) as **untrusted**

- [ ] Treat any text returned by downstream systems (logs, tickets, web pages) as **untrusted**. It can carry prompt-injection instructions.
- [ ] Redact secrets before they leave the service layer.
- [ ] Emit audit logs and Micrometer metrics for every tool call, resource read and prompt use.
- [ ] Rate-limit expensive tools.
- [ ] Keep each server's surface small and role-focused.

12. Testing

Layer	Test approach
Service logic	Plain unit tests (`DeploymentService`), including the idempotent repeat and the invalid approval case
Tool adapters	Verify parameter defaults, caps and mapping with Mockito
Resources	Assert redaction, size limits and MIME type
Prompts	Snapshot-test the generated text, and evaluate outputs against sample incidents
Whole server	Integration test with an MCP client against the running app (`@SpringBootTest`), checking the list of tools, resources and prompts

13. Link to Your CCAR-F Study Notes

CCAR-F topic	Where it shows up here
Tool descriptions drive routing	Descriptions in `@McpTool` and `@McpToolParam` say what, when and when not
Strict tool use and schemas	Typed parameters and bounded inputs keep calls well formed
Error design	Structured error: stable code, message, context, suggested action
Retry-safe patterns	Idempotency key on `restart_deployment`, circuit breaker around Kubernetes
Untrusted tool output	Logs treated as data, not instructions
Tool distribution across agents	Small, role-specific servers instead of one giant tool surface

14. Quick Reference

- **Need to act?** Tool. **Need to read?** Resource. **Need a repeatable workflow?** Prompt.
- Tools are model-controlled, resources are application-controlled, prompts are user-controlled.
- Keep tools narrow, resources scoped, and prompts explicit about arguments and output.